Deep Learning in Practice

Practical session 4: Small data and Deep Learning

MVA 2021-2022

Elias Masquil: eliasmasquil@gmail.com

Nicolas Violante: nviolante96@gmail.com

In all cases we train a ResNet-18 architecture. Our reference, trained on the full CIFAR dataset, are the ResNet-18 results reported in https://github.com/kuangliu/pytorch-cifar

Model	**Number of epochs**	**Train accuracy**	**Test accuracy**
Baseline	10	0.73	0.21
scheduler + more epochs	100	1.0	0.24
Pre-trained (32 CIFAR image size, linear training)	100	0.93	0.24
Pre-trained (32 CIFAR image size, full training)	200	0.99	0.27
Pre-trained (224 ImageNet image size, linear training)	100	1.0	0.52
Pre-trained (224 ImageNet image size, full training)	200	1.0	0.57
Data augmentation from scratch	250	0.79	0.31
Data augmentation + Pre-trained, linear training	100	0.53	0.49
Data augmentation + Pre-trained full training	200	0.96	**0.58**
Full data reference			0.932

Scheduler

We added a learning rate scheduler for better convergence of the training process. We found that the training process is more gradual and smooth by using a cosine scheduler.

Pre-trained model

Using a model pre-trained on ImageNet yields a huge gap in performance (+36%). However, we need to be careful in order to get that bust in performance. Even though the neural network can input images of any size, its weights were learned using 224×224 images of ImageNet. For this reason, we need to resize our 32×32 CIFAR images to 224×224. Directly fine-tunning the pre-trained model on 32×32 images only gives a performance increase of +3%.

We also see that it's better to train the whole model after training the last linear layer. By only training the last linear layer we get +31%. This is already a huge gain at a very cheap cost: only training one layer. But if from this point we continue training the whole model, we achieve the final +36% increase.

Of course, this strategy is available only if someone else has already trained a model in a similar dataset, which isn't always the case. Luckly, for vision tasks, ImageNet pre-trained models are widely popular.

Data augmentation

We use the same data augmentation strategy that *"Bootstrap your Own Latent"*. We observe that the data augmentation prevents the model of quickly overfitting the training dataset and get a +10% increase in performance.

However, not all problems admit the same kind of augmentations, so its application remains problem-specific. For example, on the MNIST data we can't include random flips because we would mix the 6 and 9 images. Furthermore, depending on the size of the image, some techniques might corrupt them too much (e.g rotations on tiny images).

Finally, by using data augmentation and a pre-trained model we gain a total increase of +37% of performance, for a final accuracy of **58%**. Considering that we trained the model with only 100 images from CIFAR, we find this results to be quite good.

[Bonus] Weak supervision

We consider a self-training strategy (gradually enlarging the training set by using as labels predictions from the model). Unfortunately, this approach didn't improve the performance. As usual, there are many factors that might play a role in this method (e.g how many data points add, frequency of adding those points, learning rate schedule,

etc). It’s possible that the current configuration was not appropriate, but with more care, this method might improve the results a bit.